Unit 1: Introduction to Java Programming 4 hours

1.1 History and Features of Java
1.2 C++ vs Java
1.3 Introduction
    1.3.1 JDK
    1.3.2 JVM
    1.3.3 Bytecode
    1.3.4 Java Environment.
1.4 Procedure Oriented Vs Object Oriented
1.5 Basic Concepts of Object-Oriented Programming
1.6 Structure of Java Programming, Naming conventions
1.7 Compiling and executing simple program
1.8 Scanner class

1.1 History and Features of Java

History of Java

Java's history is as interesting as it is impactful. The journey of this powerful programming language began in 1991 when James Gosling, Mike Sheridan, and Patrick Naughton, a team of engineers at Sun Microsystems known as the "Green Team," set out to create a new language initially called "Oak." Oak was later renamed Java, inspired by Java coffee, and was first publicly released in 1996 as Java 1.0. This initial version provided a no-cost runtime environment across popular platforms, making it accessible to a broad audience. Arthur Van Hoff rewrote the Java 1.0 compiler to strictly comply with its specifications, ensuring its reliability and cross-platform capabilities.

Java evolved over time, with Java 2 introducing multiple configurations tailored for different platforms, showcasing its versatility.

In 1997, Sun Microsystems aimed to formalize Java through the ISO standards body but eventually withdrew from the process.

Despite not formalizing through ISO, Sun Microsystems offered most Java implementations at no cost, earning revenue by licensing specialized products such as the Java Enterprise System.

A significant milestone in Java's history occurred on November 13, 2006, when Sun Microsystems released a large portion of the Java Virtual Machine (JVM) as free, open-source software.

By May 8, 2007, the core JVM code was fully available under open-source distribution terms.

Java was designed with core principles: simplicity, robustness, security, high performance, portability, multi-threading, and dynamic interpretation. These principles have made Java a preferred language for various applications, including mobile devices, internet programming, gaming, and e-business.

Today, Java continues to be a cornerstone of modern software development, widely used across industries and platforms.

Features of Java

1. Platform Independent

Compiler converts source code to byte code and then the JVM executes the bytecode generated by the compiler. This byte code can run on any platform be it Windows, Linux, or macOS which means if we compile a program on Windows, then we can run it on Linux and vice versa. Each operating system has a different JVM, but the output produced by all the OS is the same after the execution of the byte code. That is why we call java a platform-independent language.

2. Object-Oriented Programming

Java is an object-oriented language, promoting the use of objects and classes. Organizing the program in the terms of a collection of objects is a way of object-oriented programming, each of which represents an instance of the class.

The four main concepts of Object-Oriented programming are:

- Abstraction
- Encapsulation
- Inheritance
- Polymorphism

3. Simplicity

Java's syntax is simple and easy to learn, especially for those familiar with C or C++. It eliminates complex features like pointers and multiple inheritances, making it easier to write, debug, and maintain code.

4. Robustness

Java language is robust which means reliable. It is developed in such a way that it puts a lot of effort into checking errors as early as possible, that is why the java compiler is able to detect even those errors that are not easy to detect by another programming language. The main features of java that make it robust are garbage collection, exception handling, and memory allocation.

5. Security

In java, we don't have pointers, so we cannot access out-of-bound arrays i.e it shows ArrayIndexOutOfBounds Exception if we try to do so. That's why several security flaws like stack corruption or buffer overflow are impossible to exploit in Java. Also, java programs run in an environment that is independent of the os(operating system) environment which makes java programs more secure.

6. Distributed

We can create distributed applications using the java programming language. Remote Method Invocation and Enterprise Java Beans are used for creating distributed applications in java. The

java programs can be easily distributed on one or more systems that are connected to each other through an internet connection.

7. Multithreading

Java supports multithreading, enabling the concurrent execution of multiple parts of a program. This feature is particularly useful for applications that require high performance, such as games and real-time simulations.

8. Portability

As we know, java code written on one machine can be run on another machine. The platform-independent feature of java in which its platform-independent bytecode can be taken to any platform for execution makes java portable. WORA (Write Once Run Anywhere) makes java application to generates a '.class' file that corresponds to our applications(program) but contains code in binary format. It provides ease t architecture-neutral ease as bytecode is not dependent on any machine architecture. It is the primary reason java is used in the enterprising IT industry globally worldwide.

9. High Performance

Java architecture is defined in such a way that it reduces overhead during the runtime and at sometimes java uses Just in Time (JIT) compiler where the compiler compiles code on-demand basis where it only compiles those methods that are called making applications to execute faster.

1.2 C++ vs Java

1.3 Introduction

1.3.1 JDK

JDK (Java Development Kit)

The JDK is a software development kit that contains everything in the JRE, plus tools such as compiler, debugger, Javadoc…for developing and running Java programs or applications.

JDK is a platform-specific software and that's why we have separate installers for Windows, Mac, and Unix systems.

1.3.2 JVM

JVM (Java Virtual Machine)

When you run the Java program, the Java compiler first compiles your Java code to byte code. Then, the JVM translates byte code into native machine code. JVM is an instance of JRE, and provides an interface that does not depend on the underlying operation system and machine hardware.

1.3.3 Bytecode

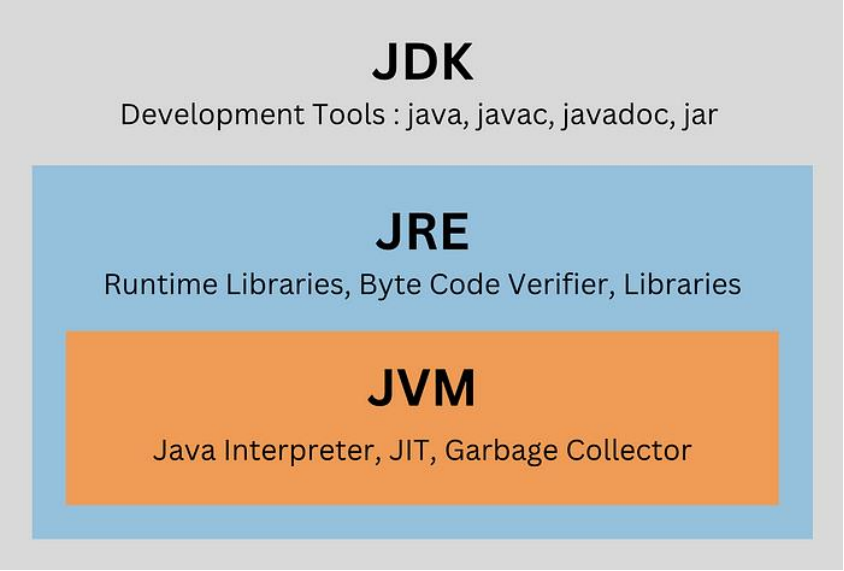
Bytecode is an intermediary form of code, such as Java bytecode, that is generated from the source code. It needs to be further converted into machine code before it can be executed on the CPU.

1.3.4 Java Environment.

JRE (Java Runtime Environment)

JRE provides the libraries, the JVM, and other components to run applets and applications written in the Java programming language.

If you just want to execute a java program, you can install only JRE. You don't need JDK because there is no development or compilation of java source code is required.



1.4 Procedure Oriented Vs Object Oriented

Procedural Programming

Procedural Programming can be defined as a programming model which is derived from structured programming, based upon the concept of calling procedure. Procedures, also known as routines, subroutines or functions, simply consist of a series of computational steps to be carried out. During a program's execution, any given procedure might be called at any point, including by other procedures or itself.

Languages used in Procedural Programming:

FORTRAN, ALGOL, COBOL, BASIC, Pascal and C.

Object-Oriented Programming

Object-oriented programming can be defined as a programming model which is based upon the concept of objects. Objects contain data in the form of attributes and code in the form of methods. In object-oriented programming, computer programs are designed using the concept

of objects that interact with the real world. Object-oriented programming languages are various but the most popular ones are class-based, meaning that objects are instances of classes, which also determine their types.

Languages used in Object-Oriented Programming:

Java, C++, C#, Python, PHP, JavaScript, Ruby, Perl, Objective-C, Dart, Swift, Scala.

Some of the differences between procedural and object-oriented programming are as:

Procedural Oriented Programming	**Object-Oriented Programming**
In procedural programming, the program is divided into small parts called ***functions***.	In object-oriented programming, the program is divided into small parts called ***objects***.
Procedural programming follows a ***top-down approach***.	Object-oriented programming follows a ***bottom-up approach***.
There is no access specifier in procedural programming.	Object-oriented programming has access specifiers like private, public, protected, etc.
Adding new data and functions is not easy.	Adding new data and function is easy.
Procedural programming does not have any proper way of hiding data so it is ***less secure***.	Object-oriented programming provides data hiding so it is ***more secure***.
In procedural programming, overloading is not possible.	Overloading is possible in object-oriented programming.
In procedural programming, there is no concept of data hiding and inheritance.	In object-oriented programming, the concept of data hiding and inheritance is used.
In procedural programming, the function is more important than the data.	In object-oriented programming, data is more important than function.

Procedural Oriented Programming	Object-Oriented Programming
Procedural programming is based on the ***unreal world***.	Object-oriented programming is based on the ***real world***.
Procedural programming is used for designing medium-sized programs.	Object-oriented programming is used for designing large and complex programs.
Procedural programming uses the concept of procedure abstraction.	Object-oriented programming uses the concept of data abstraction.
Code reusability absent in procedural programming,	Code reusability present in object-oriented programming.
Examples: C, FORTRAN, Pascal, Basic, etc.	**Examples:** C++, Java, Python, C#, etc.

1.5 Basic Concepts of Object-Oriented Programming

Java is a class-based object-oriented programming (OOP) language built around the concept of objects. OOP concepts are intended to improve code readability and reusability by defining how to structure your Java program efficiently. There are seven core principles of object-oriented programming, as follows.

List of OOP concepts in Java:

- Abstraction
- Encapsulation
- Inheritance
- Polymorphism
- Association
- Aggregation
- Composition

1.6 Structure of Java Programming, Naming conventions

A typical structure of a Java program contains the following elements:

- Documentation Section (Optional)
- Package Declaration (Optional)

- Import Statements
- Interface Section (Optional)
- Class Definition
- Class Variables and Variables
- Main Method Class
- Methods and Behaviours

Documentation Section

It includes basic information about a Java program. The information includes the author's name, date of creation, version, program name, company name, and description of the program. It improves the readability of the program. Whatever we write in the documentation section, the Java compiler ignores the statements during the execution of the program.

The comments may **be single-line**, **multi-line**, and **documentation** comments.

Single-line Comment: It starts with a pair of forwarding slash (//). For example:

//First Java Program

Multi-line Comment: It starts with a /* and ends with */. We write between these two symbols. For example:

/*It is an example of

multiline comment*/

Documentation Comment: It starts with the delimiter (/**) and ends with */. For example:

/**It is an example of documentation comment*/

Package Declaration

A java package is a group of similar types of classes, interfaces and sub-packages. Package in java can be categorized in two form, built-in package and user-defined package. There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.

Here, we will have the detailed learning of creating and using user-defined packages. For all these classes package belongs to a single parent directory. We use the keyword package to declare the package name. For example:

package nast; //where nast is the package name

package np.edu.nast; //where np is the root directory and edu is the subdirectory and nast is further sub-directory

Import Statements

The package contains the many predefined classes and interfaces. If we want to use any class of a particular package, we need to import that class. The import statement represents the class stored in the other package. It is written before the class declaration and after the package statement. We use the import statement in two ways, either import a specific class or import all classes of a particular package. In a Java program, we can use multiple import statements. For example:

import java.util.Scanner; //it imports the Scanner class only

import java.util.*; //it imports all the class of the java.util package

Interface Section

We use the interface keyword to create an interface. An interface is a slightly different from the class. It contains only constants and method declarations. Another difference is that it cannot be instantiated. We can use interface in classes by using the implements keyword. An interface can also be used with other interfaces by using the extends keyword. For example:

```
interface Car  {
void start();
void stop();
}
```

Class Definition

We use the class keyword to define the class. The class is a blueprint of a Java program. It contains information about user-defined methods, variables, and constants. Every Java program must has at least one class that contains the main() method. For example:

```
class Student {
//variables
//methods
}
```

Class Variables and Constants

We define variables and constants that are to be used later in the program. The variables and constants store values of the parameters are defined just after the class definition which is used during the execution of the program. We can also decide and define the scope of variables by using the modifiers. It defines the life of the variables. For example:

```
class Student {//class definition
        String fName; //variable
        int id;
        double percentage;
}
```

Main Method

We have to define the main() method which is essential for all Java programs. It is important because the execution of all Java programs starts from the main() method. It is also called an entry point of the class. It must be inside the class. Inside the main method, we create objects and call the methods.

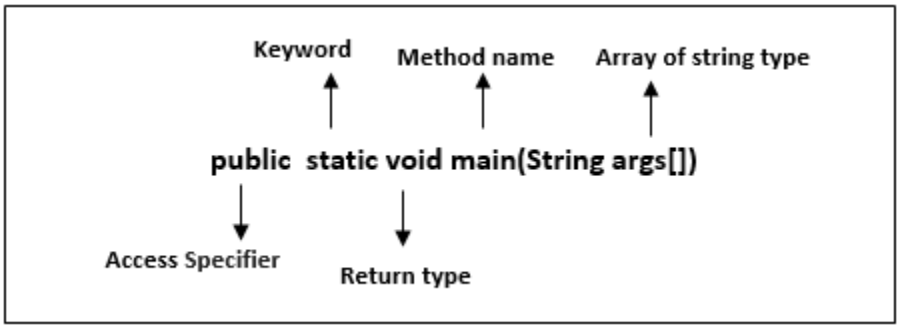
We use the following statement to define the main() method:

For example:

```
public class Student { //class definition
public static void main(String args[])  {
        //statements
}
}
```

Java main() method

The main() is the starting point for JVM to start execution of a Java program. Without the main() method, JVM will not execute the program. The syntax of the main() method is:



public: It is an access specifier. We should use a public keyword before the main() method so that JVM can identify the execution point of the program. If we use private, protected, and default before the main() method, it will not be visible to JVM.

static: You can make a method static by using the keyword static. We should call the main() method without creating an object. Static methods are the method which invokes without creating the objects, so we do not need any object to call the main() method.

void: In Java, every method has the return type. Void keyword acknowledges the compiler that main() method does not return any value.

main(): It is a default signature which is predefined in the JVM. It is called by JVM to execute a program line by line and end the execution after completion of this method. We can also overload the main() method.

String args[]: The main() method also accepts some data from the user. It accepts a group of strings, which is called a string array. It is used to hold the command line arguments in the form of string values.

main(String args[])

Here, agrs[] is the array name, and it is of String type. It means that it can store a group of string. Remember, this array can also store a group of numbers but in the form of string only. Values passed to the main() method is called arguments. These arguments are stored into args[] array, so the name args[] is generally used for it.

Methods and Behaviour

We define the functionality of the program by using the methods. The methods are the set of instructions that we want to perform. These instructions execute at runtime and perform the specified task. For example:

```
public class Demo { //class definition

public static void main(String args[])  {

        //create object and call method

}

void display()  {
```

```
        //statements
        System.out.println("Welcome to NAST");
}
}
```

Naming Conventions

Naming conventions make programs more understandable by making them easier to read. They can also give information about the function of the identifier-for example, whether it's a constant, package, or class-which can be helpful in understanding the code.

Packages

The prefix of a unique package name is always written in all-lowercase ASCII letters and should be one of the top-level domain names, currently com, edu, gov, mil, net, org, or one of the English two-letter codes identifying countries as specified in ISO Standard 3166, 1981.

Subsequent components of the package name vary according to an organization's own internal naming conventions. Such conventions might specify that certain directory name components be division, department, project, machine, or login names.

com.sun.eng

com.apple.quicktime.v2

edu.cmu.cs.bovik.cheese

Classes

Class names should be nouns, in mixed case with the first letter of each internal word capitalized. Try to keep your class names simple and descriptive. Use whole words-avoid acronyms and abbreviations (unless the abbreviation is much more widely used than the long form, such as URL or HTML).

class Raster;

class ImageSprite;

Interfaces

Interface names should be capitalized like class names.

interface RasterDelegate;

interface Storing;

Methods

Methods should be verbs, in mixed case with the first letter lowercase, with the first letter of each internal word capitalized.

run();

runFast();

getBackground();

Variables:

Except for variables, all instance, class, and class constants are in mixed case with a lowercase first letter. Internal words start with capital letters. Variable names should not start with underscore _ or dollar sign $ characters, even though both are allowed.

Variable names should be short yet meaningful. The choice of a variable name should be mnemonic- that is, designed to indicate to the casual observer the intent of its use. One-character variable names should be avoided except for temporary "throwaway" variables. Common names for temporary variables are i, j, k, m, and n for integers; c, d, and e for characters.

```
int     i;
char     c;
float    myWidth;
```

Constants

The names of variables declared class constants and of ANSI constants should be all uppercase with words separated by underscores ("_"). (ANSI constants should be avoided, for ease of debugging.)

static final int MIN_WIDTH = 4;

static final int MAX_WIDTH = 999;

static final int GET_THE_CPU = 1;

1.7 Compiling and executing simple program

Setting Up Your Computer

Before you can write and run the simple Java program you need to install the Java platform on your computer system.

The Java platform is available free of charge from the Java web site. You can choose between the Java® 2 Platform software for Windows 95/98/NT or for Solaris. The download page contains the information you need to install and configure the Java platform for writing and running Java programs.

Note: Make sure you have the Java platform installed and configured for your system before you try to write and run the simple program presented next.

Writing a Program

The easiest way to write a simple program is with a text editor. So, using the text editor of your choice, create a text file with the following text, and be sure to name the text file ExampleProgram.java. Java programs are case sensitive, so if you type the code in yourself, pay particular attention to the capitalization.

```
//A Very Simple Example

class ExampleProgram {

  public static void main(String[] args){

    System.out.println("I'm a Simple Program");

  }

}
```

Here is the ExampleProgram.java source code file if you do not want to type the program text in yourself.

Compiling the Program

A program has to be converted to a form the Java VM can understand so any computer with a Java VM can interpret and run the program. Compiling a Java program means taking the programmer-readable text in your program file (also called source code) and converting it to bytecodes, which are platform-independent instructions for the Java VM.

The Java compiler is invoked at the command line on Unix and DOS shell operating systems as follows:

```
javac ExampleProgram.java
```

Note: Part of the configuration process for setting up the Java platform is setting the class path. The class path can be set using either the -classpath option with the javac compiler command and java interpreter command, or by setting the CLASSPATH environment variable. You need to

set the class path to point to the directory where the ExampleProgram class is so the compiler and interpreter commands can find it.

Interpreting and Running the Program

Once your program successfully compiles into Java bytecodes, you can interpret and run applications on any Java VM, or interpret and run applets in any Web browser with a Java VM built in such as Netscape or Internet Explorer. Interpreting and running a Java program means invoking the Java VM byte code interpreter, which converts the Java byte codes to platform-dependent machine codes so your computer can understand and run the program.

The Java interpreter is invoked at the command line on Unix and DOS shell operating systems as follows:

java ExampleProgram

At the command line, you should see:

I'm a Simple Program

1.8 Scanner class

Scanner is a class in java.util package used for obtaining the input of the primitive types like int, double, etc. and strings.

Using the Scanner class in Java is the easiest way to read input in a Java program, though not very efficient if you want an input method for scenarios where time is a constraint like in competitive programming.

Java Scanner Input Types

Scanner class helps to take the standard input stream in Java. So, we need some methods to extract data from the stream. Methods used for extracting data are mentioned below:

Method	Description
nextBoolean()	Used for reading Boolean value
nextByte()	Used for reading Byte value
nextDouble()	Used for reading Double value
nextFloat()	Used for reading Float value
nextInt()	Used for reading Int value
nextLine()	Used for reading Line value
nextLong()	Used for reading Long value
nextShort()	Used for reading Short value

```
// Java program to read data of various types
// using Scanner class.
import java.util.Scanner;

// PersonInfo Class
public class PersonInfo {
	// main function
	public static void main(String[] args)
	{
		// Declare the object and initialize with
		// predefined standard input object
		Scanner sc = new Scanner(System.in);

		// String input
		String name = sc.nextLine();

		// Character input
		char gender = sc.next().charAt(0);

		// Numerical data input
		// byte, short and float can be read
		// using similar-named functions.
		int age = sc.nextInt();
		long mobileNo = sc.nextLong();
		double cgpa = sc.nextDouble();

		// Print the values to check if the input was
		// correctly obtained.
		System.out.println("Name: " + name);
```

```
            System.out.println("Gender: " + gender);
            System.out.println("Age: " + age);
            System.out.println("Mobile Number: " + mobileNo);
            System.out.println("CGPA: " + cgpa);
        }
}
```